



SOFTWARE RE-ENGINEERING

FINAL PROJECT

Re-Engineering a Legacy Hospital Management System

Department of Software Engineering

Total Marks: 100 | BS Software Engineering

Field	Details
Assessment Type	Group Project — 2 Members
Submission Format	PDF Report + SQL Scripts + GitHub Repository
Viva	Required — tools and database must be live during viva
Plagiarism Policy	Zero marks for the entire group
Late Submission	Penalty applies as per course policy

Group Members

Member	Full Name	Roll Number
1		
2		

Marks Distribution at a Glance

Section	Max Marks	Marks Obtained
Part A — Project Initialisation and Tool Setup	8	
Part B — Code Smell Analysis and Refactoring	27	
Part C — Dependency, Coupling and Technical Debt	15	
Part D — Dynamic Program Analysis	10	
Part E — Data Smell Detection	15	
Part F — Schema Normalisation and Refactoring	15	
Part G — Data Migration Design and Execution	10	
TOTAL	100	

Note	Code Smell Analysis (Part B) and Database parts (Parts E, F, G) together carry 67 out of 100 marks — reflecting their central importance in software re-engineering.
------	--

Project Background

HealthBridge Hospital has operated a legacy patient management system for fifteen years. Built as a monolithic Java application with a partially relational, partially flat-file database, the system was extended by multiple development teams with no architectural discipline. The codebase now suffers from pervasive code smells that violate object-oriented principles, while the database has accumulated data smells that cause reporting errors and integrity failures.

Your team has been engaged as software re-engineering consultants. You will apply a full re-engineering pipeline — selecting a real Java project for code analysis, and working with the provided legacy hospital schema for all database tasks. Every finding must be supported by actual tool output, real SQL, and your own written interpretation.

The Two Artefacts You Will Work On

Artefact	What It Is	Used In
Open-Source Java Project	A Java project you select from GitHub (criteria in Part A)	Parts A, B, C, D
Legacy Hospital DB Schema	The schema provided in Part E of this document	Parts E, F, G

Part A — Project Initialisation and Tool Setup [8 Marks]

A1. Java Project Selection [3 Marks]

Select one open-source Java project from GitHub satisfying all criteria below. You will use this same project throughout Parts B, C, and D.

- Minimum 2000 lines of application source code (excluding tests and auto-generated files)
- At least six distinct Java classes with inter-class dependencies
- Contains at least one long method, one large class, or one obvious structural problem visible to the naked eye — these will be the focus of Part B
- Has not been used in any previous coursework by either group member
- Compiles successfully on your machine with no build errors

In your report document:

1. Provide the full GitHub URL and write two paragraphs describing what the system does
2. State the total line count, class count, and Java version
3. Include a screenshot of the repository page with your account or fork visible
4. Explain in one paragraph why this project is a good candidate for code smell analysis

A2. Tool Installation and Verification [5 Marks]

Tool	Purpose	Verification Evidence Required
SonarQube via Docker	Static analysis, smell detection, metrics	Screenshot of dashboard at localhost:9000
SonarScanner	Submits project to SonarQube	Terminal output showing BUILD SUCCESS
Docker	Hosts SonarQube container	docker --version output
Python Tutor (online)	Step-by-step execution tracing	Screenshot with code loaded
Draw.io or Graphviz	Control flow and dependency diagrams	Screenshot of a diagram open in the tool
MySQL or PostgreSQL	Run the legacy hospital schema	mysql --version or psql --version

For each tool, include the version number alongside the screenshot. Screenshots without version numbers or console output receive half marks.

Part B — Code Smell Analysis and Refactoring [27 Marks]

Code smells are surface indicators of deeper design problems. They do not prevent a program from running, but they slow development, introduce bugs, and make maintenance expensive. This part requires you to identify real code smells in your chosen project, classify them precisely, explain their root cause, and propose concrete refactoring treatments.

Weight

Part B carries 27 marks — the largest single section — because identifying, classifying, and reasoning about code smells is the core skill of software re-engineering.

B1. SonarQube Analysis and Metrics Extraction [5 Marks]

Configure and execute SonarScanner against your project. Include the sonar-project.properties file you used, a screenshot of the SonarScanner terminal output, and a screenshot of the SonarQube project overview dashboard.

Extract all metrics below. For each, write two to three sentences of project-specific interpretation — generic definitions copied from lecture slides will not receive marks.

Metric	Your Value	Your Project-Specific Interpretation
Lines of Code (LOC)		
Total Code Smells		
Cyclomatic Complexity (total)		
Average CC per function		
Cognitive Complexity		
Code Duplication (%)		
Maintainability Rating (A–E)		
Technical Debt (hours)		
Security Hotspots		

B2. The Five Code Smell Categories — Deep Identification [16 Marks]

Your selected project must be examined through the lens of all five official smell categories. For each category, identify at least two real smells from your codebase. Every smell entry must include the exact file name and line number so it can be verified during viva.

Category 1: Bloaters [4 Marks]

Bloaters are code elements that have grown so large they are hard to work with. They accumulate gradually as programs evolve and are rarely cleaned up. The five Bloater smells are: Long Method, Large Class, Long Parameter List, Primitive Obsession, and Data Clumps.

Reference definitions:

- Long Method — A method with excessive lines of code where scrolling is required to read it. All logic crammed into one function makes it tightly coupled and fragile. Treatment: Extract Method.
- Large Class — A class trying to do too much, violating the Single Responsibility Principle. As size grows, cohesion drops. Treatment: Extract Class into focused responsibilities.
- Long Parameter List — A method signature with so many parameters it becomes hard to call correctly and reveals that the class is pulling data from too many places. Treatment: Introduce Parameter Object.
- Primitive Obsession — Using primitive data types (String, int, float) to represent rich domain concepts instead of small dedicated classes. Treatment: Replace Data Value with Object.
- Data Clumps — The same group of two to four variables appearing together across multiple methods or classes, signalling a missing abstraction. Treatment: Extract Class or Introduce Parameter Object.

Smell Name	File and Line Number	Evidence — what you see in the code	Why It Is a Bloater	Recommended Treatment
(e.g. Long Method)				
(e.g. Large Class)				

For the Long Method or Large Class you found, paste the method or class signature (not the full body) in your report and annotate it to show exactly which lines make it a smell.

Category 2: Object-Orientation Abusers [3 Marks]

These smells result from incomplete or incorrect application of object-oriented principles. The four OO-Abuser smells are: Switch Statements, Temporary Fields, Refused Bequest, and Alternative Classes with Different Interfaces.

- Switch Statements — A switch or chain of if-else that selects behaviour based on a type field. Every time a new type is added, every switch must be updated. Treatment: Replace with Polymorphism (Strategy or State pattern).
- Temporary Fields — A class instance variable that is only set and used in one specific scenario, leaving it null or meaningless the rest of the time. Treatment: Extract Class or Introduce Null Object.
- Refused Bequest — A subclass inherits methods from a parent but ignores or overrides most of them, failing the IS-A test and violating the Liskov Substitution Principle. Treatment: Replace Inheritance with Delegation or restructure the hierarchy.
- Alternative Classes with Different Interfaces — Two or more classes that do the same thing but have different method names or signatures, preventing them from being used interchangeably. Treatment: Rename Methods or Extract Superclass / common interface.

Smell Name	File and Line	Evidence from Your Code	OO Principle Violated	Treatment

Category 3: Change Preventors [3 Marks]

Change Preventors mean that a single logical change forces modifications in many unrelated places, making maintenance expensive and error-prone. The three smells are: Divergent Change, Shotgun Surgery, and Parallel Inheritance Hierarchies.

- Divergent Change — One class must be edited for multiple unrelated reasons (e.g., a change to the reporting format AND a change to the data validation logic both require editing the same class). This violates SRP. Treatment: Extract Class to separate responsibilities.
- Shotgun Surgery — A single logical change (e.g., changing a tax rate) requires small edits scattered across many classes. Treatment: Move Method and Move Field to consolidate related behaviour.
- Parallel Inheritance Hierarchies — Every time a subclass is added to one hierarchy, a corresponding subclass must be added to another. Treatment: Merge hierarchies or use delegation.

Smell Name	File(s) and Lines	How Many Places Must Change?	Treatment Strategy

Category 4: Dispensables [3 Marks]

Dispensables are things that serve no purpose — their removal would make the code cleaner, smaller, and easier to read. The five dispensable smells are: Comments (excessive/redundant), Duplicate Code, Lazy Class, Data Class, and Speculative Generality.

- Comments (Excessive or Redundant) — Comments that describe what the code does rather than why, repeating what clear naming already communicates. They become a maintenance burden. Comments are sometimes called the deodorant of code — masking a bad smell instead of removing it. Treatment: Rename methods and variables to be self-documenting; remove redundant comments.
- Duplicate Code — The same logic appears in two or more places. Fixing a bug in one copy but not the others introduces subtle inconsistencies. Treatment: Extract Method and call it from both sites.
- Lazy Class — A class that does almost nothing, typically just delegating to one other class. It adds navigation overhead with no benefit. Treatment: Inline Class or Collapse Hierarchy.
- Data Class — A class with only fields, getters, and setters, no meaningful behaviour. Objects should encapsulate both data and behaviour. Treatment: Move Method to bring behaviour to the data.

- Speculative Generality — Code written for hypothetical future requirements that do not exist yet (gold plating). Unused abstract classes, hook methods, or parameters. Treatment: Collapse Hierarchy, Inline Class, or remove unused code.

Smell Name	File and Line	What Makes It Dispensable?	Treatment

Category 5: Couplers [3 Marks]

Couplers represent excessive or inappropriate coupling between classes. The five coupler smells are: Feature Envy, Inappropriate Intimacy, Message Chain, Middleman, and Incomplete Library Class.

- Feature Envy — A method that spends more time using data or methods of another class than its own. The intelligence is in the wrong place. Treatment: Move Method to the class whose data it envies.
- Inappropriate Intimacy — Two classes that know too much about each other's private details, often because a class was split incorrectly. Treatment: Move Method/Field, or Encapsulate Field to establish proper boundaries.
- Message Chain — A client calls object A to get B, then B to get C, then C to get D. The client is tightly coupled to the entire chain. Treatment: Hide Delegate or extract the chain into a dedicated method.
- Middleman — A class whose only purpose is to delegate every call to another class. If more than half of its methods simply forward to another object, it adds no value. Treatment: Remove Middleman and call the delegated object directly.
- Incomplete Library Class — A library class that almost does what you need but is missing one or two methods, forcing you to add logic in client code or subclass just to add the missing behaviour.

Smell Name	File and Line	Description of the Coupling Problem	Treatment

B3. Smell Interaction and Prioritisation [3 Marks]

Real systems rarely have isolated smells — one smell often causes another. In your report, write a structured analysis of 250 words or more addressing:

5. Select any two smells from different categories that you found in the same area of your project. Explain how one smell likely contributed to or caused the other.
6. Of all the smells you identified across the five categories, which single smell poses the greatest risk to the long-term maintainability of this project? Justify your answer with reference to the specific code location and what a future developer would face when trying to modify that area.

7. Which refactoring treatment would you apply first, and why? Consider both the effort required and the benefit delivered.

B4. Refactoring Demonstration [3 Marks]

Choose one smell from your analysis — preferably one in Category 1 or 5 — and demonstrate the refactoring in code.

8. Paste the original code snippet (the smelly version) with the affected lines highlighted or annotated
9. Apply the appropriate refactoring treatment and paste the improved version
10. Write a paragraph confirming that the external behaviour is unchanged and explaining what structural property was improved

Viva Note

During viva you will be asked to point to the original smell in your project and explain your refactoring live. Both group members must be able to answer questions on any smell from any category.

Part C — Dependency, Coupling and Technical Debt [15 Marks]

C1. Dependency Mapping [7 Marks]

Select six classes from your project. For each, determine Ca (afferent coupling — how many classes depend on it), Ce (efferent coupling — how many classes it depends on), and Instability using the formula below.

Afferent Coupling

Ce = classes outside this module that depend ON it

Efferent Coupling

Ce = classes outside this module that this module depends ON

Instability

I = Ce / (Ca + Ce)

I = 0.0 --> Maximally stable (many dependants, few dependencies)

I = 1.0 --> Maximally unstable (no dependants, many dependencies)

Class Name	Ca	Ce	Instability (I)	Stable / Volatile	Key Observation

Show full working for at least two classes: list by name each class that contributes to Ca and Ce. Then draw the dependency graph using Draw.io or Graphviz and embed it in your report. Identify any circular dependency or God Class (Ce > 5).

C2. Technical Debt Assessment [8 Marks]

Identify three real technical debt items from your Java project. Each must reference an actual file and line number. Classify each item and compute the remediation cost.

Debt Classification

Item	File + Line	Debt Type	Intentional?	Prudent or Reckless?
D1			Yes / No	Prudent / Reckless
D2			Yes / No	Prudent / Reckless
D3			Yes / No	Prudent / Reckless

Debt types: Design Debt, Code Debt, Test Debt, Documentation Debt, Dependency Debt, Architecture Debt.

Remediation Cost Calculation

Step 1: Extract SonarQube remediation estimate for each item (minutes)
Step 2: Add 25% contingency buffer --> Buffered = Raw x 1.25
Step 3: Total Remediation Effort = sum of all buffered values
Step 4: Total Development Effort = SonarQube LOC x 30 minutes per line
Step 5: Debt Ratio (%) = (Total Remediation / Total Development) x 100

Interpretation: 0-5% = Healthy 5-10% = Moderate >10% = Risky

Item	Raw Estimate (min)	+ 25% Buffer (min)	Buffered Total (min)
D1			
D2			
D3			
TOTAL			

After computing the Debt Ratio, state the health category and write a 200-word prioritisation argument: which debt item would you fix first and why, considering business impact, cost, and deferral risk.

Part D — Dynamic Program Analysis [10 Marks]

Select one non-trivial method from your Java project that contains at least one conditional statement and at least four local variables. Use it for all three tasks below.

D1. Execution Trace with Python Tutor [4 Marks]

Load the method (or a self-contained adaptation of it) into Python Tutor and trace it step by step. In your report:

11. Paste the full source code of the chosen method and name the class it belongs to
12. Complete the execution trace table below
13. Include a Python Tutor screenshot at the moment a conditional branch is evaluated, with the frame panel showing variable values
14. Write one paragraph explaining which branch was taken and why, based on the actual variable state

Step	Statement Executed	Variables Before	Variables After	Notes
1				
2				
3				
4				
...	Continue as needed			

D2. Control Flow Graph [3 Marks]

Draw the CFG of the same method using Draw.io or Graphviz. Requirements:

- Each node is a basic block; each edge is a control transfer
- Decision nodes labelled with the boolean condition
- The path taken in D1 highlighted with a different colour or bold border
- One ENTRY node and at least one EXIT node

After embedding the diagram, answer: (1) How many independent paths does the method have? (2) Calculate Cyclomatic Complexity using $CC = E - N + 2$. Compare with SonarQube's value for the same method. (3) What one change would reduce CC by 1?

D3. Abstract Syntax Tree Inspection [3 Marks]

Load the same method into AST Explorer (astexplorer.net) with the Java parser. In your report:

15. Embed a screenshot of the AST expanded to at least three levels

16. Annotate the following node types on the screenshot or in the text: MethodDeclaration, VariableDeclaration, IfStatement (or SwitchStatement), ReturnStatement, BinaryExpression
17. Explain what the IfStatement node's internal structure reveals about how Java represents branching. Describe one practical re-engineering use case for ASTs — for example in a refactoring engine, a linter, or a code generator.

Prisma Usage Requirement:

All database-related tasks (Parts E, F, and G) must be implemented using Prisma ORM.

Instructions:

- 1. Install Prisma using: `npm install prisma --save-dev`
- 2. Initialize Prisma: `npx prisma init`
- 3. Configure database connection in `.env` file
- 4. Define schema in `schema.prisma` reflecting the legacy and refactored structures
- 5. Run migrations using: `npx prisma migrate dev`
- 6. Use Prisma Client for querying and validation in migration scripts
- 7. Include screenshots of `schema.prisma`, migration logs, and sample queries

Part E — Data Smell Detection [15 Marks]

Weight

Parts E, F, and G together cover the entire database layer and carry 40 marks — more than any other domain in this project — reflecting the central role of data restructuring in legacy system re-engineering.

The legacy HealthBridge Hospital Management System stores data in the MySQL schema below. This schema was written incrementally over fifteen years by multiple developers and has never been formally reviewed. Your task is to read it carefully, identify data smells, and document them with precision.

E1. The Legacy Hospital Schema

```
-- HealthBridge Hospital Management System — Legacy Schema
-- Accumulated 2009-2024. Never formally reviewed.

CREATE TABLE pat_master (
  pid          INT,
  p_name       VARCHAR(255),
  dob          VARCHAR(50),      -- stored as 'DD/MM/YYYY' plain text
  sex          CHAR(1),          -- 'M', 'F', or '3' for non-binary
  ph1          VARCHAR(255),
  ph2          VARCHAR(255),
  ph3          VARCHAR(255),      -- repeating phone group
  addr1        VARCHAR(255),
  addr2        VARCHAR(255),
  city         VARCHAR(255),
  reg_doc      VARCHAR(255),      -- doctor full name stored as plain text
  reg_doc_id   VARCHAR(255),      -- sometimes INT string, sometimes 'DR-042'
  total_visits INT,              -- updated manually, not via trigger
  last_bill    FLOAT,           -- stores PKR amounts; FLOAT used for
  currency     TEXT,
  notes        TEXT              -- JSON blobs, free text, and CSV tags all
  mixed
);
```

```

CREATE TABLE appointments (
  appt_id      INT,
  patient_id   INT,
  patient_nm   VARCHAR(255),           -- duplicated from pat_master
  patient_ph   VARCHAR(255),           -- duplicated from pat_master
  doc_id       INT,
  doc_name     VARCHAR(255),           -- duplicated from doctors table
  appt_date    VARCHAR(50),           -- 'YYYY-MM-DD HH:MM' stored as text
  status       CHAR(1),                -- 'P'=Pending 'C'=Complete 'X'=Cancel
  'H'=Hold 'R'=Rescheduled
  fee          FLOAT,
  discount     FLOAT,
  net_fee      FLOAT,                 -- always = fee - discount (derived value)
  room         VARCHAR(255)           -- 'Room 3 Block B' - two facts in one column
);

CREATE TABLE doctors (
  DoctorID     INT PRIMARY KEY,
  FullName     VARCHAR(255),
  Speciality   VARCHAR(255),
  ContactNo    VARCHAR(255),
  JoinDt       VARCHAR(50),           -- date stored as text
  Salary       FLOAT,                -- monthly salary stored as FLOAT
  dept_id      INT,                  -- references departments but no FK defined
  isActive     CHAR(1)               -- 'Y', 'N', or sometimes '1'
);

CREATE TABLE billing (
  bill_no      VARCHAR(50),           -- intended as PK but no constraint defined
  pid          INT,
  pname        VARCHAR(255),           -- patient name duplicated again
  services     TEXT,                  -- 'Lab,Xray,OPD' - comma-separated list
  svc_cost     FLOAT,
  tax_pct      FLOAT,
  tax_amt      FLOAT,                 -- derived: svc_cost * tax_pct / 100
  grand_total  FLOAT,                 -- derived: svc_cost + tax_amt
  paid         FLOAT,
  balance      FLOAT,                 -- derived: grand_total - paid
  created      VARCHAR(50),           -- date stored as text
  created_by   VARCHAR(255)           -- username as free text; no FK to users
);

CREATE TABLE departments (
  dept_id      INT PRIMARY KEY,
  dept_nm      VARCHAR(255),
  hod          VARCHAR(255),           -- head-of-department stored as plain name
  budget       FLOAT
);

```

E2. Smell Identification — Complete Reference Table [9 Marks]

Identify at least ten distinct data smells from the schema. Each smell must come from a different table-column combination. Use the official smell categories from the course lectures. The table must be fully completed — partially filled rows receive no marks.

#	Table	Column(s)	Smell Category	Smell Name	Evidence from Schema	Real-World Risk in a Hospital	Proposed Fix
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							

Smell category reference — use these exact category names in your table:

- Redundancy: Duplicate Data, Derived Data
- Structural: Unnormalized Table, Overloaded Column, Non-Atomic Fields, God Table
- Naming: Inconsistent Naming, Ambiguous Column Names
- Integrity: Missing Keys/Constraints, Hidden Relationships
- Data Type: Type Optimization Smell, Misused Boolean Flag
- Semantic: Magic Values / Encoded Nulls, Inconsistent Units, Polymorphic Association
- Security and Lifecycle: Lack of Audit Trail, Sensitive Data Unencrypted, No Retention Policy

E3. Smell Prioritisation and Business Justification [6 Marks]

Of the ten smells you identified, choose the four that pose the greatest risk to a live hospital system. For each, write a concrete patient-care or regulatory scenario that illustrates the consequence of leaving the smell unaddressed. Then rank all four and justify the ranking.

Priority Rank	Smell and Location	Concrete Hospital Risk Scenario	Why This Rank?
1st			
2nd			

Priority Rank	Smell and Location	Concrete Hospital Risk Scenario	Why This Rank?
3rd			
4th			

After completing the table, write a paragraph (150 words minimum) explaining the connection between data smell severity and patient safety risk in a hospital information system. Use at least two of your identified smells as examples.

Part F — Schema Normalisation and Refactoring [15 Marks]

F1. Normalisation of pat_master up to 3NF [6 Marks]

The pat_master table violates multiple normal forms. Apply normalisation step by step, documenting each violation and the transformation applied.

Step 1 — Identify Violations

Normal Form	Violated? (Yes/No)	Specific Violation Example from pat_master
1NF — Atomic values; no repeating groups		
2NF — Full dependency on the whole key		
3NF — No transitive dependencies		

Step 2 — Write the Normalised Schema (CREATE TABLE statements)

Write complete, runnable CREATE TABLE statements for your normalised design. Every table must have a clearly defined PRIMARY KEY. Foreign keys must reference the correct parent table. Use appropriate data types — no VARCHAR for dates, no FLOAT for monetary amounts.

```
-- Write your normalised CREATE TABLE statements below.
-- Minimum expected tables: patients, patient_phones, patient_addresses
-- Additional tables may be needed depending on your analysis.

CREATE TABLE patients (
  -- your columns here
);

CREATE TABLE patient_phones (
  -- your columns here
);

-- add further tables as required by your normalisation
```

Step 3 — Before and After Comparison

Aspect	Before (pat_master)	After (Normalised Design)
Number of tables	1	

Aspect	Before (pat_master)	After (Normalised Design)
Repeating phone columns	ph1, ph2, ph3	
Address storage	Inline addr1/addr2/city	
Doctor reference	Name as plain text	
Date of birth column type	VARCHAR(50)	
Primary key	None defined	
Currency columns	FLOAT	

F2. Five Schema Refactoring Scripts [6 Marks]

Apply the five refactoring operations below to the legacy schema. For each, write a runnable SQL script and follow it with two to three sentences explaining the smell it fixes and the risk it eliminates.

Refactoring R1 — Fix Derived Data in billing (1.5 Marks)

Remove tax_amt, grand_total, and balance from the billing table. Replace with a view that computes them on read.

```
-- R1: Remove derived columns from billing
ALTER TABLE billing DROP COLUMN tax_amt;
ALTER TABLE billing DROP COLUMN grand_total;
ALTER TABLE billing DROP COLUMN balance;

-- Create view to compute derived values on read
CREATE OR REPLACE VIEW v_billing_summary AS
SELECT
    bill_no,
    pid,
    svc_cost,
    tax_pct,
    ROUND(svc_cost * tax_pct / 100, 2)           AS tax_amt,
    ROUND(svc_cost + svc_cost * tax_pct / 100, 2) AS grand_total,
    paid,
    ROUND(svc_cost + svc_cost * tax_pct / 100 - paid, 2) AS balance
FROM billing;
```

[Write your explanation of the Derived Data smell and how this view eliminates it here.]

Refactoring R2 — Fix Overloaded Column in appointments.status (1 Mark)

Replace the opaque CHAR(1) status code with a reference table and enforce it via a foreign key.

```
CREATE TABLE appt_status_ref (
    status_code CHAR(1) PRIMARY KEY,
```

```

        description VARCHAR(50) NOT NULL
    );
INSERT INTO appt_status_ref VALUES
    ('P','Pending'), ('C','Completed'), ('X','Cancelled'),
    ('H','On Hold'), ('R','Rescheduled');

ALTER TABLE appointments
    ADD CONSTRAINT fk_appt_status
    FOREIGN KEY (status) REFERENCES appt_status_ref(status_code);

```

[Write your explanation here.]

Refactoring R3 — Fix Inconsistent Naming across doctors (1 Mark)

Standardise all column names to lowercase snake_case in the doctors table.

```

ALTER TABLE doctors RENAME COLUMN DoctorID TO doctor_id;
ALTER TABLE doctors RENAME COLUMN FullName TO full_name;
ALTER TABLE doctors RENAME COLUMN Speciality TO speciality;
ALTER TABLE doctors RENAME COLUMN ContactNo TO contact_no;
ALTER TABLE doctors RENAME COLUMN JoinDt TO join_date;
ALTER TABLE doctors RENAME COLUMN Salary TO salary_monthly;
ALTER TABLE doctors RENAME COLUMN isActive TO is_active;

-- Apply the same standard to any other tables with mixed casing.
-- Document every renamed column below:

```

[List any additional tables you also renamed and state the naming convention adopted.]

Refactoring R4 — Fix Missing Constraints in billing and appointments (1 Mark)

Add the missing primary key to billing and the missing foreign keys to both billing and appointments. Include a data-backfill step to remove orphan rows before enabling the FK.

```

-- Step 1: Add PK to billing
ALTER TABLE billing ADD PRIMARY KEY (bill_no);

-- Step 2: Remove orphan billing rows that reference non-existent patients
DELETE FROM billing WHERE pid NOT IN (SELECT pid FROM pat_master);

-- Step 3: Add FK from billing to pat_master
ALTER TABLE billing
    ADD CONSTRAINT fk_billing_patient
    FOREIGN KEY (pid) REFERENCES pat_master(pid);

-- Step 4: Add FK from appointments to doctors
ALTER TABLE appointments
    ADD CONSTRAINT fk_appt_doctor
    FOREIGN KEY (doc_id) REFERENCES doctors(DoctorID);

```

[Explain why the DELETE backfill step is necessary before the FK constraint can be applied.]

Refactoring R5 — Add Audit Trail to appointments (0.5 Mark)

```
ALTER TABLE appointments
  ADD COLUMN created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  ADD COLUMN updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                          ON UPDATE CURRENT_TIMESTAMP;

-- For PostgreSQL, use an explicit trigger:
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN  NEW.updated_at = NOW();  RETURN NEW;  END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_appt_audit
BEFORE UPDATE ON appointments
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

[Explain the Lack of Audit Trail smell and the hospital-specific risk it creates.]

F3. Refactoring Impact Summary [3 Marks]

Refactoring	Smell(s) Resolved	Tables Changed	Risk Eliminated	Effort vs Benefit
R1 — Derived data view	Derived Data	billing	Inconsistency in financial totals	
R2 — Status reference table	Overloaded Column	appointments	Invalid status codes	
R3 — Naming standardisation	Inconsistent Naming	doctors + others	ORM errors, dev confusion	
R4 — PK/FK constraints	Missing Constraints	billing, appointments	Orphan rows, silent corruption	
R5 — Audit trail	Lack of Audit Trail	appointments	No accountability for changes	

Write a paragraph evaluating which of the five refactorings delivered the greatest quality improvement per unit of effort, and justify your answer using specific evidence from the schema.

Part G — Data Migration Design and Execution [10 Marks]

The HealthBridge team needs to migrate legacy appointment records from a CSV export produced by the old system into the newly refactored relational schema from Part F. This part simulates the Extract-Transform-Load pipeline that would precede a production cutover.

G1. Legacy Source Data

The old system exported appointment records in the CSV format below. This is your migration source.

```
appt_id,patient_id,patient_nm,patient_ph,doc_id,doc_name,appt_date,status,fee,discount,net_fee,room
1001,5,"Ali Hassan","0312-9876543",12,"Dr. Kamran Raza","15/03/2024
09:30","P",1500.00,0.00,1500.00,"Room 3 Block B"
1002,8,"Sara Malik","0333-1234567",7,"Dr. Ayesha Noor","15/03/2024
10:00","C",2000.00,200.00,1800.00,"Room 7 Block A"
1003,5,"Ali Hassan","0312-9876543",12,"Dr. Kamran Raza","16/03/2024
11:00","X",1500.00,0.00,1500.00,"Room 3 Block B"
1004,21,"Hina Iqbal","0321-5556789",7,"Dr. Ayesha Noor","17/03/2024
09:00","H",2000.00,500.00,1500.00,"Room 7 Block A"
... (assume 500 rows total in the real file)
```

G2. Migration Plan [3 Marks]

Plan Element	Your Response
Source format	CSV exported from legacy system
Target schema	Refactored appointments table (from Part F)
Estimated row count	~500 records
Required transformations (list at least 4)	
Columns to drop (redundant after normalisation)	
ETL tool or language	
Rollback strategy if migration fails	
Validation method after migration	
Estimated execution time	
Will the system require downtime?	

G3. ETL Transformation Script [5 Marks]

Write a complete Python or SQL migration script implementing all four required transformations below. Include inline comments on every non-trivial line.

#	Transformation	Reason (Data Smell Addressed)
T1	Convert appt_date from 'DD/MM/YYYY HH:MM' string to a proper DATETIME value	Data Type smell — dates stored as text
T2	Split the room column ('Room 3 Block B') into room_number INT and building_block VARCHAR	Non-Atomic Fields smell — two facts in one column
T3	Omit patient_nm, patient_ph, doc_name from the INSERT — they are now derived from FK lookups	Duplicate Data smell — redundant after normalisation
T4	Validate each status value against appt_status_ref; log and skip any row with an unknown code	Overloaded Column fix — invalid codes must not enter the new schema

```
# migration_etl.py — Load legacy CSV into refactored appointments schema
import csv, mysql.connector
from datetime import datetime

VALID_STATUSES = {'P', 'C', 'X', 'H', 'R'}

def parse_appt_date(row):          # T1: 'DD/MM/YYYY HH:MM' --> datetime
    # implement here
    pass

def split_room(row):              # T2: 'Room 3 Block B' --> (3, 'Block B')
    # implement here
    pass

def migrate(csv_path, db_config):
    conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor()
    skipped = []

    with open(csv_path, newline='', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:

            # T4: validate status; skip and log unknown codes
            if row['status'] not in VALID_STATUSES:
                skipped.append(row['appt_id'])
                continue

            appt_dt = parse_appt_date(row['appt_date']) # T1
            room_no, block = split_room(row['room'])    # T2

            # T3: patient_nm, patient_ph, doc_name intentionally omitted
            cursor.execute(
```

```

        '''INSERT INTO appointments
        (appt_id, patient_id, doc_id, appt_datetime,
        status, fee, discount, room_number, building_block)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)'''
        (row['appt_id'], row['patient_id'], row['doc_id'],
        appt_dt, row['status'], row['fee'], row['discount'],
        room_no, block)

    )

    conn.commit()
    print(f'Done. Skipped {len(skipped)} rows with invalid status: {skipped}')
    cursor.close(); conn.close()

# Complete the pass stubs above, run the script, and include a terminal
screenshot.

```

Complete all pass stubs, run the script against your locally loaded CSV sample (minimum 10 rows), and include a screenshot of the terminal showing the migration output with row counts.

G4. Post-Migration Validation [2 Marks]

Run all four queries below against the migrated data and record results. If any check fails, describe your investigation and resolution.

```

-- V1: Row count must match valid rows in CSV
SELECT COUNT(*) AS migrated_rows FROM appointments;

-- V2: No NULL datetime values (T1 must have worked for every row)
SELECT COUNT(*) AS null_dates FROM appointments WHERE appt_datetime IS NULL;
-- Expected: 0

-- V3: Only valid status codes exist (T4 must have filtered invalid codes)
SELECT DISTINCT status FROM appointments;
-- Expected: only codes present in appt_status_ref

-- V4: No orphan appointments (patient_id must exist in patients table)
SELECT COUNT(*) AS orphans FROM appointments a
LEFT JOIN patients p ON a.patient_id = p.patient_id
WHERE p.patient_id IS NULL;
-- Expected: 0

```

Query	Expected	Your Result	Pass/Fail	Action Taken if Fail
V1 — Row count	Matches valid CSV rows			
V2 — Null dates	0			

Query	Expected	Your Result	Pass/Fail	Action Taken if Fail
V3 — Valid statuses	P,C,X,H,R only			
V4 — No orphans	0			

Submission Requirements and Academic Integrity

What to Submit

- One PDF report containing all parts A through G, properly sectioned and paginated with figure labels on every screenshot
- A GitHub repository containing: the forked or cloned Java project, sonar-project.properties, all SQL scripts from Parts F and G, the Python migration script, and a README with setup instructions
- The README must include commands to start SonarQube, run the scanner, load the hospital schema, and execute the migration script

Report Standards

- Every screenshot must be followed immediately by a written explanation. Screenshots without explanation receive zero marks.
- All SQL scripts must be runnable without modification. Scripts with syntax errors receive zero marks for that section.
- All interpretations must be specific to your chosen project and the provided schema. Generic definitions copied from lecture slides receive zero marks.
- Tables must be fully completed. Rows left blank or filled with placeholder text receive no marks.
- Both group members must attend the viva. SonarQube, the database, and the migration script must be in a running state on the presenting machine.

Academic Integrity

Each group's Java project will differ. The database analysis is performed on the same provided schema, but your smell identification, normalisation decisions, refactoring scripts, and written analyses must reflect your own reasoning. Groups found to have copied written analysis from one another will receive zero marks for the entire project. All plagiarism cases are reported to the department.

Final Marks Summary

Section	Max Marks	Marks Obtained
Part A — Project Initialisation and Tool Setup	8	
Part B — Code Smell Analysis and Refactoring	27	
Part C — Dependency, Coupling and Technical Debt	15	
Part D — Dynamic Program Analysis	10	
Part E — Data Smell Detection	15	

Section	Max Marks	Marks Obtained
Part F — Schema Normalisation and Refactoring	15	
Part G — Data Migration Design and Execution	10	
TOTAL	100	